PAL Architecture and Design

This document explains PAL's organization, the reasons behind its interfaces, and its implementation trade-offs. It focuses on execution architecture, state semantics, and subsystem design rather than teaching the API from the beginning. Use the API Guide for runnable modeling workflows and public API behavior, the API Reference for exact signatures, and the Cheatsheet for quick lookup.

Contents

1. Foundations: responsibilities and execution — How PAL's Python, compiled, and native parts work together to run a model.
2. State, ownership, and execution semantics — Where model data lives, how changes take effect, and what safe and fast modes mean.
3. Subsystem design choices — Why PAL uses different kinds of grids, agent collections, random sampling, and visualization tools.

1. Foundations: responsibilities and execution

1.1 The design problem PAL is solving

PAL targets models that repeatedly update many small pieces of state. A timestep might inspect thousands of neighborhoods, move individual agents, update a population count at every site, solve a field equation, or combine several of these operations. Expressing every inner operation as a Python call is flexible but can incur substantial interpreter overhead. Writing the whole model in C could reduce that overhead, but would make iteration and experimentation less convenient for Python users.

PAL's architecture splits the problem by frequency and responsibility. Python remains the control language for constructing the model, selecting parameters, loading inputs, managing experiments, and deciding when to draw or checkpoint. Repeated computation is expressed in functions that PAL can prepare for Numba compilation. The native core supplies low-level state and kernels behind that interface.

This division lets a high-level model combine different state representations without rewriting its timestep in C, while allowing low-level implementations to evolve without changing the model-facing API. This requires coordinated maintenance of wrappers, compiled overloads, and native state. This split does not mean that all Python code is slow or that every model function should be compiled. It puts optimization effort where repeated work makes it worthwhile. A function called once during setup usually needs less attention than a loop executed at every site over millions of timesteps. Start with clear model code, identify the repeated work, and benchmark before moving more logic into compiled kernels.

1.2 The layers and their responsibilities

PAL's architecture can be understood through six cooperating responsibilities:

1. **Public API and Protocol declarations.** These define the names, signatures, type-facing interface, and expected user-visible operations. They support readable calls, type annotations, and autocomplete.
2. **Python constructors and wrappers.** These create objects, translate public arguments into implementation calls, and expose native-backed state through a Python-friendly interface.
3. **Numba-compatible wrappers and overloads.** These let compiled model functions call PAL operations while preserving supported argument forms and return behavior.

4. **The `pal.njit` preparation step.** This transforms selected PAL-specific constructs before compilation, including neighborhood/box iteration and some source-aware diagnostics.
5. **The native C core.** This provides low-level storage and computational routines used by the Python/Numba-facing layer.
6. **Tests, examples, and generated documentation.** These act as executable and written statements of the public contract, checking that the layers agree.

These are responsibilities rather than necessarily separate packages; several may share a source file. Typically, model code calls a public wrapper, which selects a Python or Numba-compatible implementation. For compiled functions, `pal.njit` may transform supported PAL-specific loops before Numba compilation. The resulting operation accesses native-backed state or a native kernel where appropriate. Not every operation passes through every layer. For example, `pal.njit` is relevant to compiled model functions that use PAL's supported syntax; it is not a mandatory step for ordinary Python calls. Keeping the responsibilities conceptually distinct helps localize defects. Constructor bugs, Numba typing errors, native-state problems, and stale API declarations require different fixes.

1.3 The native core and Python interface

PAL implements low-level operations in its native core without requiring users to write C for ordinary model logic. Native-backed objects can hold state in a representation suited to repeated updates, while the Python-facing API offers recognizable constructors, indexing, and methods.

The native implementation is not itself the public contract. A C function may accept a particular pointer or integer layout internally, but the Python API should expose a stable and understandable operation. The wrapper layer is where public arguments, supported types, safe-mode validation, and implementation details meet.

This separation creates a maintenance obligation: behavior must remain coherent across the wrapper and native layers. If the Python method accepts a keyword argument but its Numba overload silently binds that keyword differently, the layers disagree. If a native routine handles a one-dimensional coordinate differently from the safe wrapper, a mode-specific defect can result. PAL's parity tests therefore compare outcomes across paths rather than assuming that one layer automatically inherits the correctness of another.

1.4 The public API is a contract, not a mirror of the C implementation

PAL's public names and signatures are designed around modeler-facing concepts: a grid site, an agent handle, a population count, a neighborhood, a field, or a random draw. The native core may use lower-level representations that do not resemble those concepts directly. That is acceptable as long as the public behavior remains consistent.

Protocol declarations are part of this contract because users and tools read them for annotations and autocomplete. But a declaration is not executable proof. Likewise, a docstring or manual paragraph cannot make unsupported runtime behavior valid. The implementation, declarations, tests, and generated references must describe the same public API.

API changes therefore require coordination across layers. When altering an argument name, return type, indexing rule, or safe-mode behavior, check the Python method, the Numba overload, relevant Protocol declaration, documentation generator, examples, and regression tests. A change that works in one path but breaks another is not complete merely because the primary example still runs.

1.5 Python and Numba execution paths

Python execution is useful for interactive exploration, setup code, inspection, and small operations. Numba-compiled execution is useful for repeated loops where Python dispatch would be costly. PAL supports both because a scientific workflow naturally mixes these kinds of work.

Supporting both introduces complexity. Numba imposes typing and compilation constraints that ordinary Python does not. Native-backed objects need compatible wrappers or overloads; argument types and names must be represented in forms Numba understands; and code transformations must produce valid compilable functions. An operation can therefore work in Python and fail at compile time, or compile successfully but behave differently in an edge case.

PAL treats parity as a correctness requirement for supported usage, not as a promise that Python and compiled code have identical internals or performance. Compare observable behavior: return values, mutations, errors where they are part of the contract, boundary handling, and state after the operation. If a difference appears, reduce it to a minimal reproducer and test both paths.

1.6 Model-code transformation with `pal.njit`

PAL's `njit` interface is not simply a spelling alias for Numba's decorator. Before compilation, it can prepare functions that use PAL-specific syntax and diagnostics. In particular, neighborhood and Box iteration are model-friendly abstractions that may need to be expanded into forms that the compiler can type and execute efficiently.

The transformation lets users express spatial operations without manually converting coordinates in every loop. It also gives PAL a place to add source-aware safe-mode diagnostics when argument annotations provide enough type information.

This approach keeps spatial loops readable while allowing their compiled form to specialize iteration. It also introduces a translation stage with its own syntax restrictions, type assumptions, and failure modes that matter during debugging. The preparation step uses Python source inspection; if the function's source cannot be retrieved, PAL leaves the function unchanged and passes it to Numba. Consequently, PAL-specific loop rewriting should not be assumed to work for dynamically defined functions whose source is unavailable. The Python source a modeler reads is not necessarily the exact function body Numba receives. Tracebacks and typing errors may refer to transformed code or a synthetic compilation context. Annotations improve the information available to the transformer, but they do not guarantee that every error can be attributed to a perfect source location.

When a failure occurs under `@pal.njit`, first reproduce it in safe mode, without calling `pal.FastMode()`. If the additional checks do not identify the problem, temporarily remove `@pal.njit` from the functions involved, starting with the ones closest to the failure, and use a Python debugger to follow the execution. Once the error is understood, restore compilation and verify the fix in both Python and compiled execution. If the failure occurs only when compiled, reduce it to a small example and check PAL's transformation requirements, argument annotations, and supported API usage before changing the scientific algorithm.

1.7 A concrete execution trace

Consider a model step that reads a field value, changes an agent, and queues a population delta. At the **model layer**, the author chooses the order of those operations. The **public PAL API** resolves coordinates, handles, and arguments. In ordinary Python execution, Python-facing wrappers perform the supported operations; in a compiled step, PAL's preparation and Numba-compatible call path must preserve their observable meaning. Both paths ultimately work with the relevant native-backed state. A later `Update()` on the population grid commits the queued delta; it is not implied by a field read or an agent movement.

A successful API call does not establish a correct model step: dispatch, location, and update timing are separate concerns. Ask three questions when debugging: **Was the right site or handle selected? Was the right operation dispatched? Was the state committed at the intended point?** A failure in any one can produce a scientifically wrong timestep despite apparently reasonable model code.

This execution path is conceptual; not every call crosses every layer. Some operations are resolved in wrappers or compiled code; consult the actual implementation when profiling call overhead or changing a kernel.

2. State, ownership, and execution semantics

2.1 Native-backed state and object lifetime

A PAL object's public methods operate on state managed through PAL's native-backed implementation, not necessarily an ordinary Python container. The Python object is the user's handle to that state; it is not safe to assume that every internal value is an ordinary Python list or NumPy array that can be edited freely.

This distinction affects object lifetime and mutation. Creating, moving, or disposing an agent must keep position, occupancy, and handle validity consistent. Writing directly into an internal count or pointer-like representation could bypass those invariants, so PAL exposes explicit state-changing methods. Likewise, a returned array or slice must be interpreted according to its contract: a detached copy is a snapshot, not a live view into native memory, and editing it changes the copy until the result is explicitly assigned back.

Native-backed objects can support pickling, but pickling an object is not equivalent to serializing the entire simulation process. A model's complete checkpoint may also need parameters, timestep, other coupled objects, external state, and random-number state. See the API Guide's saving section for the checkpoint contract and its limitations.

2.2 Preserving invariants across layers

The public contract specifies behavior rather than pointer layout: operations must preserve invariants, and serialized state must be reconstructed without relying on process-local addresses. A wrapper can reject an invalid operation before it reaches native code, but a compiled path may reach the same underlying state through a different route. Invariants therefore need protection and tests at the boundaries where state can actually change.

When adding an operation, identify which layer owns each invariant. A population update must reconcile queued deltas with current counts and capacity constraints; a PDE transport operation must respect configured geometry, boundary treatment, and commit semantics. Do not expose a mutable native buffer without defining its ownership, lifetime, invalidation, and synchronization rules.

2.3 Relative-offset neighborhoods

A neighborhood such as Moore or Von Neumann is naturally a pattern around a location. PAL represents it as relative integer offsets, then applies it to a specific grid position with `grid.Hood(...)`. This separates the reusable pattern from the grid location where it is used.

Applying a hood must account for grid dimensions and wrapping flags; the relative pattern alone does not know whether a particular neighbor falls outside a nonperiodic domain. PAL's application step can translate offsets into coordinates, wrap periodic axes, and omit out-of-domain sites according to the public contract.

Relative offsets avoid storing absolute-neighbor lists for every cell and allow the compiled path to transform high-level neighborhood loops. Neighborhood destinations are not necessarily unique: small or wrapped domains can map multiple offsets to the same site. Whether to count those repeated contributions or deduplicate them depends on the model, so choose and test the intended behavior.

2.4 Separate update and commit operations

PAL makes a distinction between changes that are being accumulated and changes that have become the current state. For `PopGrid` and `PDEgrid`, `Add` queues a delta and `Update` commits pending changes. This exposes the timestep boundary in code.

This distinction has mathematical consequences. If every new site value must be computed from the same old field, immediate mutation can make results depend on iteration order. Queued deltas let the model calculate changes before committing them. This does not guarantee synchronous updates in every case: reads observe the current committed state, while direct indexing can mutate it immediately. PAL permits order-dependent algorithms but makes the choice explicit.

Agent updates have different semantics. `All()` snapshots which handles will be visited, but it does not snapshot the world. If one agent reproduces, later agents may see the newborn's occupancy even though the newborn is not part of the original handle snapshot. This is a deliberate and useful pattern for sequential agent models, but it is not equivalent to a synchronous cellular update.

PAL does not impose one scheduler because these update schemes represent different models. For example, a two-site population transfer can queue `Add(A, -1)` and `Add(B, 1)`, then call `Update()` once: both changes are calculated against the old committed counts. By contrast, moving an agent immediately changes which site subsequent agents encounter. The two representations can model similar populations but do not automatically implement the same update schedule. Choose the intended timing and test it with a small case whose result can be calculated by hand.

2.5 Process-wide safe and fast modes

Safe mode performs selected validation and provides diagnostics that are useful while developing a model. Fast mode removes selected checks from performance-sensitive paths. Keeping one mode active for a process makes the execution contract predictable for objects created within that process; PAL locks the choice at first construction.

A process-wide mode also avoids pretending that an individual object can be switched independently when calls across objects may share compiled functions or native machinery. The consequence is operational: call `pal.FastMode()` before creating any PAL object or window, and run safe and fast test suites in separate processes.

Fast mode should be treated as a measured optimization, not a correctness mode. It can reduce validation overhead; it cannot repair invalid model assumptions, unsafe numerical parameters, or a wrong update order. Safe mode can catch selected contract violations, but it cannot establish that the biological or physical equations are appropriate.

3. Subsystem design choices

Chapter 1 describes how a model operation reaches its implementation; Chapter 2 describes the state and timing rules that must hold regardless of execution path. This chapter examines the concrete representations that enforce those rules. For each subsystem, the key questions are what its representation makes easy and what it leaves to the modeler. For complete call signatures and usage examples, see the API Reference and API Guide. The API Guide also covers performance options, safety-mode selection, and checkpointing. Those usage instructions complement the design rationale here; they do not replace tests of implementation behavior or scientific validity.

3.1 Grid: typed site state and explicit indexing

A `Grid` represents one typed value per lattice site. The element type is chosen at construction, which makes the intended state representation explicit and allows the implementation to use compact native storage rather than a Python object at every site. Use a type that matches the model's meaning: integer labels for categories, Boolean values for masks, and floating-point values for continuous attributes.

Indexing supports both linear site indices and coordinate forms, but these are different views of the same lattice. Use coordinate indexing when the equation is naturally spatial; use a linear index when iterating sites or passing a site identifier between APIs. In more than one dimension, coordinate tuples are often easier to review because the axes remain visible. The conversions `ToI` and `ItoX` / `ItoY` / `ItoZ` are explicit so the mapping can be tested rather than guessed.

The fast compiled path has a specific optimization: scalar `Grid` indexing can lower to a direct memory load or store, avoiding a PAL function call for each site access. This is possible because the native-backed grid has a fixed element type and its compiled representation carries the data address. The trade-off is that fast scalar access bypasses some checks performed by the safe path; the model must use valid indices and representable values. This is an optimization of scalar indexing, not a promise that arbitrary NumPy slicing or Python operations become zero-overhead in compiled code.

Grid slices are detached NumPy copies, not live native-backed views. Editing the result changes only the copy; to update the grid, assign through the grid itself. For example, `region = grid[2:5, 3:8]; region[:] = 7` does not change `grid`, whereas `grid[2:5, 3:8] = 7` does. This distinction makes ownership explicit at the Python boundary. In safe mode, assignment validates relevant type, range, and nonfinite conditions; do not rely on fast mode to diagnose invalid values.

3.2 AgentGrid: handles, occupancy, and mutation

An agent handle is distinct from its position. The handle identifies the living agent; position determines where it is located. This separation lets a model retain identity while an agent moves, and lets queries report handles without copying whole Python objects. The model remains responsible for the meaning and lifecycle of any additional per-agent state. Handles may be reused after disposal, so a saved handle value is not a permanent identity across an agent's death and a later allocation. A snapshot of handles preserves the values selected for iteration, not historical identities; models that need persistent lineage identifiers should maintain them separately.

Stackability is an explicit constructor choice because allowing multiple agents at a site changes the model's state space and affects occupancy logic. A nonstackable grid rejects a second occupant in safe mode. Do not enable stackability merely to suppress an error: decide whether multiple occupancy is biologically or physically meaningful.

The distinction between `All()` and a live iteration matters for mutation. `All()` returns a snapshot of handles and is therefore the appropriate pattern when the loop can create or dispose agents. A snapshot freezes membership, not agent state or grid occupancy. Each later query observes current occupancy. If newborns must not act until the next timestep, build the snapshot before processing the timestep; if all agents must compute from the same old world, snapshot the required state as well.

The `counts` property is a read-only occupancy query view, not a writable alternate path to agent state. This is a deliberate consistency boundary: occupancy is derived from agent placement and lifecycle, not a second independently editable source of truth. Changing a count directly would bypass agent handles, placement rules, and lifecycle bookkeeping. Query occupancy through this view; create, move, and dispose agents through the AgentGrid API.

3.3 PopGrid: counts instead of individual objects

A `PopGrid` stores integer population counts per site. It is intended for populations whose members are interchangeable for the questions being asked. This is a modeling assumption, not just a memory optimization: populations with equal site counts are indistinguishable even when their individual histories differ. That can greatly reduce storage and update overhead compared with representing every individual as an agent, but it removes individual-level history and identity.

Its `Add` operation queues a delta; `Update` applies pending changes; `Reset` clears current state and pending deltas. The explicit delta buffer makes it possible to compute a set of changes before committing them. For a movement event from site A to site B, queue a negative delta at A and a positive delta at B, then update once. If the model reads current counts during this phase, it sees the pre-commit counts, which helps make the update independent of visitation order.

The `capacity` option sets a per-site population cap, not a limit on the sum across the grid. It constrains how many interchangeable individuals a single location may hold; a model that needs a global population ceiling must enforce that separately. `GetPop()` reports the total across all sites, whereas `All()` returns the indices of nonzero sites, not the counts themselves. Read the count at each returned index when its local value is needed. Keeping these semantics distinct avoids confusing the set of occupied sites with the population data stored there.

3.4 PDEgrid: numerical operations with explicit assumptions

A `PDEgrid` stores a lattice scalar field. Unlike an AgentGrid, its state is a spatially distributed quantity rather than a collection of individually tracked entities. This representation supports field-level transport and source terms, but the modeler must choose a discretization and use consistent units for stored values and coefficients. Its transport methods represent different mathematical choices: scalar-coefficient diffusion, spatially varying or interface-defined coefficients, masked interfaces, alternating-direction implicit diffusion, radial diffusion on circular or spherical domains, and constant or spatially varying advection. Choose a method based on the equation and geometry being modeled, not simply on which method is fastest.

`SetTimeSpaceStep` configures the temporal step `dt` and spatial steps `dx`, `dy`, and `dz` used by transport methods. These values are part of the discretized equation. A timestep that is too large can produce unstable or inaccurate results even when every API call is valid. Boundary conditions likewise belong to the mathematical problem: face-specific values and periodic wrapping can produce different solutions.

The distinction between queued and immediate mutations is especially important for fields. `Add()` queues a field delta for `Update()` to apply, whereas direct indexed assignment changes a site immediately. Do not assume every write is queued. Neither queued updates nor an implicit or explicit numerical method guarantees stability, positivity, or conservation on its own; those properties depend on the scheme, coefficients, timestep, and boundaries. Validate against a known field, test boundary behavior, and check convergence as timestep and grid spacing change.

3.5 IList: reusable collections for compiled workflows

`IList` stores reusable integer collections in Python and compiled code. For example, a model can collect empty neighboring sites and choose one for reproduction without allocating a new collection at every step.

`All()` returns a detached NumPy copy; `Iter()` exposes the current native-backed values without making that detached copy. This avoids allocations in tight compiled loops, but it is not a snapshot: changing the list while consuming an `Iter()` result can alter the values being traversed or invalidate assumptions about its storage. Use `All()` when iteration must be insulated from later mutations, and `Iter()` only while the underlying list remains unchanged. Clear reused lists before repopulating them, and check that a list is nonempty before calling `Random()`.

3.6 Randomness: one explicit PAL stream

`Seed`, `Random`, and `RandInt` share PAL's random stream across Python and compiled code. Each draw advances the stream, so even one additional draw changes subsequent outcomes. Reproducibility requires matching the seed, initial state, parameters, and draw order.

A shared stream helps reproduce a fixed execution order across Python and compiled calls, but also couples components through draw order. Changing when one component samples can change the outcomes of later components even when their own code is unchanged. Use PAL's RNG functions when a model should share one reproducible stream across PAL calls. External RNGs have independent state and need their own seed or checkpoint handling. A PAL object checkpoint does not by itself serialize the random stream, so exact stochastic continuation requires separately preserving or reconstructing random state. Do not claim that a seed alone guarantees identical outcomes after changing the algorithm.

3.7 Multinomial sampling: configuration versus sample state

A `Multinomial` sampler separates configuration from the state accumulated while sampling. `Setup(n)` configures the number of trials for repeated `Sample(p)` calls; `Binomial(n, p)` performs a draw with an explicit trial count. `NewMultinomial(other)` copies the sampler's configuration, but not its current multinomial remainder, so the new sampler starts with independent sampling state. Both samplers must use the same process-wide safety mode. Cloning therefore preserves the configuration without continuing a partially consumed sample.

3.8 Visualization: simulation state versus presentation

Rendering is deliberately separate from simulation state transitions. That separation permits headless runs and intermittent drawing, and it keeps visualization costs from being mistaken for model-update costs. Benchmark simulation and rendering separately before measuring end-to-end performance. A plausible-looking image does not establish that the state transitions are correct.

`PixWindow` exposes a writable pixel buffer for lattice-style images; `OpenGLWindow` draws 2D/3D geometry. They serve different rendering workflows.

Write pixel colors or OpenGL primitives, then call the window's `Update()` to publish a frame. Rendering every timestep helps inspection but may dominate runtime; long simulations can render selected steps. Keep drawing separate from the computational kernel unless using an explicitly supported compiled drawing API.

Headless rendering exists so output can be generated and tested without a visible interactive window, but it still depends on the relevant image or graphics backend. A headless test validates the supported off-screen path; it cannot fully validate window placement, user input, or a display server's lifecycle.

3.9 Compiled-loop specialization and diagnostics

PAL's `njit` wrapper transforms selected constructs, including neighborhood and Box iteration, before Numba compilation. The neighborhood pattern must be a compile-time tuple of integer-offset tuples, supplied literally or through a resolvable module or closure constant. This lets PAL specialize the loop before Numba compilation instead of interpreting an arbitrary runtime collection. The `unroll=True` option is meaningful in this compiled path: it lets PAL expand a fixed neighborhood loop so each neighbor operation is exposed to compilation. Because the loop body is duplicated into separate statements, Hood-level `break` and `continue` are unsupported with `unroll=True`; use the ordinary Hood loop when those control-flow semantics are required. Unrolling can remove loop-control overhead, but it duplicates the loop body in generated code; use it for small stencils, not large neighborhoods, where compile time and code size can outweigh runtime gains. In ordinary Python iteration, `unroll=True` does not provide this compilation-time optimization.

Python syntax alone does not guarantee matching compiled behavior: AST rewriting, wrappers, native code, and Numba typing all contribute. When a problem appears only under compilation, reduce it to a minimal function and compare Python and compiled execution.

Source-line diagnostics depend on sufficient type information, including annotations for relevant PAL arguments. Unannotated arguments do not promise the same diagnostics. Treat annotations as part of writing clear compiled model code, not as a guarantee that every error can be attributed to an exact source expression.